UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ

FACULTAD DE INGENIERÍA DE SISTEMAS

PROYECTO DE EVALUACIÓN PARCIAL

Asignatura	Tecnologías Emergentes (ISO46B)
Modalidad	Grupos de 4 estudiantes / Asignación de dataset distinto por grupo
Defensa	Semana 8
Entrega	Lunes 19/10/2026 – 23:59 hrs mediante ADESA (enlace github)
Calificación	Escala vigesimal (0–20). Ver la rúbrica adjunta.

1. ¿En qué consiste?

Durante las primeras siete semanas aprendieron, por separado, a preparar datos, entrenar modelos, evaluarlos, versionarlos, publicarlos como API y conversar con ellos mediante un chatbot. En este proyecto lo harán todo junto, sobre un dataset público que no han visto en clase.

2. Qué deben construir

ETAPA	QUÉ SE ESPERA	EVIDENCIA
E1. Planteamiento	Definir el problema, quién usaría el sistema, qué decisión apoya, qué error es más costoso y qué métrica lo refleja. Declarar en qué momento se haría la predicción y con qué datos se contaría entonces.	Sección 1 del informe
E2. Datos y exploración	Cargar, limpiar y explorar el dataset. Detectar y resolver los problemas de calidad. Dividir en entrenamiento, validación y prueba SIN fuga de información (el preprocesamiento se ajusta solo con entrenamiento).	src/datos.py · notebook · sección 2
E3. Modelado	Una línea base (modelo trivial y un modelo simple) y una red neuronal con al menos dos versiones distintas. Comparar con evidencia y concluir con honestidad, incluso si la red no es mejor.	src/modelo.py · sección 3

E4. Evaluación y versionado	Métricas adecuadas al problema, elección de la versión ganadora usando SOLO validación, evaluación final única en prueba, y una ficha (JSON) por versión con su fecha, datos, parámetros y métricas, más la promoción de la versión elegida.	src/evaluacion.py · modelos/ · sección 4
E5. API	Una API (Flask o FastAPI) que cargue la versión promovida y responda predicciones, con validación de entradas, errores claros (código 400) y un endpoint de salud.	src/api.py · sección 5
E6. Chatbot	Un chatbot que consuma la API: diseño del flujo (diagrama), recolección de datos paso a paso (slots), manejo de lo que no entiende (fallback), derivación a una persona y mensajes comprensibles para un usuario no técnico.	src/chatbot.py · diagrama · sección 6
E7. Calidad	Pruebas automáticas con pytest (datos, API y chatbot), README que permita reproducir todo desde cero y archivo requirements.txt.	tests/ · README.md
E8. Comunicación	Informe técnico breve y defensa oral con demostración en vivo.	Informe PDF · defensa

3. Adaptaciones según su dataset

Cada dataset exige ajustes. Revisen qué aplica al suyo y justifíquenlo en el informe:

- Dos clases (binaria): elijan el umbral de decisión en validación según el costo de cada tipo de error.
- Más de dos clases (multiclase): no hay un único umbral; usen macro-F1, matriz de confusión y recall por clase. La API debe devolver la clase y las probabilidades.
- Pocos datos (menos de ~1 000 filas): un solo particionado da métricas inestables. Usen validación cruzada estratificada y reporten la variabilidad.
- Desbalance de clases: la exactitud (accuracy) engaña. Comparen con una línea base trivial y usen F1, precisión, recall o PR-AUC.
- Atributos sensibles (sexo, edad, nacionalidad, estado civil, salud): decidan si se usan, justifiquen y midan si el error cambia entre grupos.
- Variables que no se conocen en el momento real de decidir: son fuga de información. Identifíquenlas y expliquen cómo las trataron.

4. Estructura obligatoria del repositorio

```
proyecto_grupoXX/
├── README.md              # cómo instalar y ejecutar todo desde cero
├── requirements.txt
├── entrenar.py            # corre todo el entrenamiento y deja los modelos en modelos/
├── src/
│   ├── datos.py           # carga, limpieza y partición
│   ├── modelo.py          # preprocesador y redes neuronales
│   ├── evaluacion.py      # métricas, versionado y promoción
│   ├── preprocesamiento_api.py   # validación de entradas de la API
│   ├── api.py             # servicio web
│   └── chatbot.py         # asistente conversacional
├── tests/                 # pruebas con pytest
├── modelos/               # versiones guardadas + fichas JSON (puede omitirse si pesa mucho)
├── docs/                  # diagrama del flujo del chatbot y figuras
├── notebooks/             # exploración y análisis (notebook ejecutado)
└── informe.pdf
```

5. Qué entregar

N.°	ENTREGABLE	FORMATO
1	Repositorio completo con la estructura de la sección 4	Carpeta comprimida (.zip) o enlace a repositorio
2	Informe técnico (máximo 8 páginas, sin contar anexos) con las secciones 1 a 6 del cuadro de etapas, una sección 7 de «Limitaciones y ética» y una sección 8 de «Contribución de cada integrante»	PDF
3	Diagrama del flujo del chatbot (caminos feliz, alternativo y de error)	PNG o PDF dentro de docs/
4	Declaración de uso de herramientas de IA (qué se usó, para qué, y qué se verificó a mano)	Una página en el informe
5	Presentación de apoyo para la defensa (opcional, máximo 8 diapositivas)	PPTX o PDF

6. Defensa

MOMENTO	DURACIÓN	QUÉ OCURRE
Presentación	8 minutos	El grupo explica el problema, las decisiones clave, los resultados y las limitaciones.
Demostración en vivo	4 minutos	Se muestra la API respondiendo y el chatbot conversando, incluyendo un caso de error.
Preguntas individuales	8 minutos	El docente pregunta a cada integrante, incluidos temas de partes que no desarrolló. Las respuestas cuentan para la nota individual.

7. Reglas de integridad

- Pueden consultar apuntes, documentación y herramientas de IA, pero deben declarar su uso. Código que no pueden explicar no cuenta como aprendido.
- Cada grupo trabaja su propio dataset. Compartir código entre grupos, o copiarlo de un repositorio ajeno, se considera falta de integridad académica.
- Los resultados deben ser los que su código produce. Inventar o retocar métricas invalida la entrega.
- Usar el conjunto de prueba para decidir (elegir modelo, umbral o parámetros) invalida la evaluación final: la prueba se mira UNA sola vez.
- El historial del repositorio (commits) se usa para verificar la contribución de cada integrante.

8. Consejos

- Un sistema sencillo que funciona de punta a punta vale más que uno ambicioso que no se puede ejecutar.
- Empiecen por una línea base: si un modelo trivial ya obtiene un resultado parecido, es importante saberlo.
- Guarden cada versión del modelo con su ficha desde el primer día; no lo dejen para el final.
- Prueben el chatbot con frases mal escritas, respuestas fuera de rango y personas que se equivocan: allí se nota el diseño.
- Hagan commits pequeños y frecuentes, con mensajes claros.